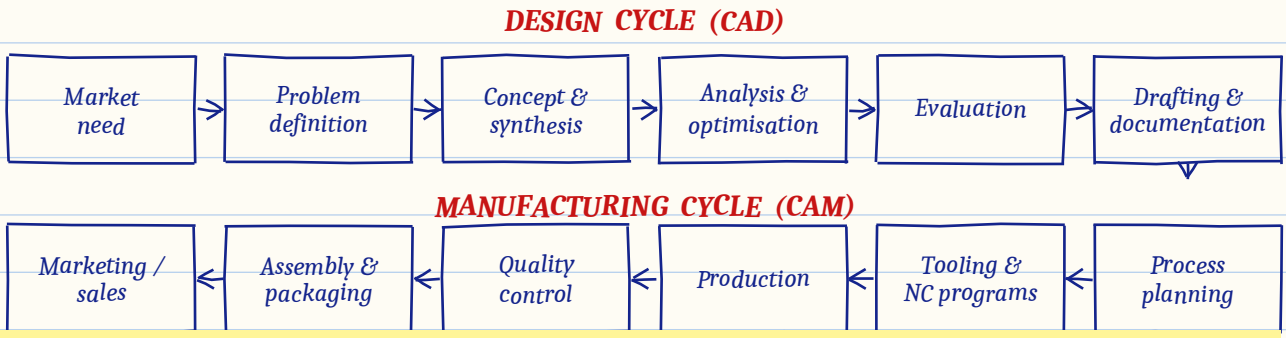
UNIT – I : Overview of CAD/CAM & Computer Graphics

1. Product Cycle

- **Product cycle** = all the activities from the customer's need / idea of a product till it reaches the customer (and its disposal).
- It is divided into two halves:
  - **Design cycle** (office / CAD side): market need → problem definition → conceptual design → analysis & optimisation → evaluation → drawings & documentation.
  - **Manufacturing cycle** (shop-floor / CAM side): process planning → tooling & NC programs → production → quality control → assembly → packaging → marketing & sales.
- In the conventional (manual) cycle the two halves are separate, so information is re-typed, redrawn and misunderstood → long lead time and errors.
- With computers a **common database** is shared by both halves, so the cycle is shorter and cheaper.



Concurrent / simultaneous engineering: design and manufacturing teams work in parallel on the same data instead of one after the other.

2. CAD – Computer Aided Design

- **CAD** is the use of computer systems to assist in the creation, modification, analysis or optimisation of a design. (Groover)
- Hardware: computer (CPU), graphics display (CRT / LCD), input devices (keyboard, mouse, light pen, digitiser, scanner), output devices (plotter, printer).

- Software: modelling package + analysis package + database.

Design process and where CAD helps (Shigley's 4 phases)

- **Geometric modelling** – describe the shape mathematically (wire-frame, surface, solid).
- **Engineering analysis** – FEA, stress, heat transfer, mass properties, kinematics, vibration.
- **Design review & evaluation** – interference checking, animation, simulation, tolerance check.
- **Automated drafting** – 2D views, sections, dimensioning, bill of materials, drawing standards.

Benefits of CAD

- Higher designer productivity (3 to 10 times) and shorter design time.
- Better design quality – more alternatives can be analysed.
- Easy modification, standardised drawings, better communication.
- Creates a manufacturing database (geometry, dimensions, BOM, tolerances) that CAM can reuse directly.

3. CAM – Computer Aided Manufacturing

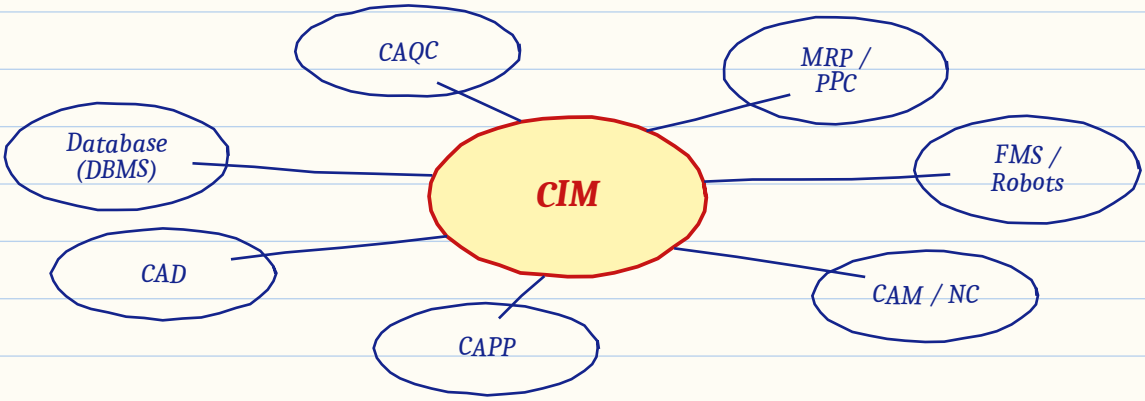
- **CAM** = use of computer systems to plan, manage and control the operations of a manufacturing plant through direct or indirect computer interface with the plant's production resources.
- Two broad areas:
  - **Computer monitoring and control** – the computer is connected directly to the process (NC / CNC machines, robots, process control).
  - **Manufacturing support applications** – the computer is used indirectly: process planning, part programming, production scheduling, inventory, MRP, shop-floor control.
- Examples: NC part programming, CNC & DNC, robot programming, automated inspection (CMM), AGV and AS/RS, FMS.

Benefits of CAM

- Reduced lead time, scrap and rework; consistent quality.
- Better machine utilisation, flexible production, lower inventory.
- Less skilled-labour dependence, better safety.

4. CIM – Computer Integrated Manufacturing

- **CIM** = integration of all business and manufacturing functions (design, planning, production, quality, marketing, accounts) through a **common computer database** and network. (CASA/SME wheel)
- CIM = CAD + CAM + business functions (production planning & control, MRP, costing, sales).
- Goal: "right information at the right place at the right time" → agile, low-cost, high-quality production.



CAD/CAM vs CIM

Feature	CAD / CAM	CIM
Scope	design + manufacturing only	whole enterprise incl. business & marketing
Data	may be separate islands	one shared database
Aim	automate individual tasks	integrate all tasks
Benefit	faster design & production	shortest lead time, max flexibility

5. CAD Tools and CAM Tools

CAD tools (software / functions)

- **Drafting tools**: AutoCAD, DraftSight – 2D drawing, layers, dimensioning, hatching.
- **3D modelling tools**: CATIA, Creo (Pro/E), SolidWorks, NX (Unigraphics), Autodesk Inventor.
- **Analysis tools**: ANSYS, Abaqus, NASTRAN – FEA; Adams – mechanisms.
- **Utilities**: data exchange (IGES, STEP, DXF), rendering, animation, tolerance analysis, PDM.

CAM tools

- **NC programming systems**: Mastercam, NX CAM, PowerMill, Fusion 360 CAM, CATIA machining, APT.
- **Process planning (CAPP) software**, tool-path simulation and verification, post-processors.
- **Shop-floor tools**: DNC software, MES, MRP/ERP packages, CMM and inspection software, robot simulators.

6. Utilisation of CAD/CAM in an Industrial Environment

- Aerospace and automobile – body surface design, crash analysis, die & mould making.
- Machine-tool and capital goods – design of parts, assemblies and NC machining of complex parts.
- Electronics – PCB layout, chip design (VLSI), automated assembly.
- Architecture & construction (CAAD), shipbuilding, plant layout, piping design.
- Tool, die & mould shops – 3 to 5 axis machining from CAD solid models.
- Typical implementation steps in a company:
  - study present system and set objectives → select hardware / software (use evaluation criteria)
  - plan installation & training → pilot project → full implementation → review benefits.

7. Evaluation Criteria for a CAD/CAM System

- **Functional**: modelling capability (2D / 3D / solid / surface), drafting, analysis, NC module, data exchange (IGES / STEP).
- **Hardware**: CPU speed, RAM, graphics card, display resolution, storage, plotter.
- **Software**: user-friendliness (GUI), customisation (macros, API, programming language), database, vendor support.
- **Cost**: initial cost + training + maintenance + upgrades (total life-cycle cost).
- **Compatibility** with existing systems, networking ability, expansion / upgrade path.
- **Vendor**: reputation, service, documentation, user base and training.
- **Performance**: response time, regeneration speed, accuracy, reliability.
- Final decision usually by a **weighted-score (rating) method** – each criterion given a

weight, each system scored, total compared.

8. CAD Data Structure

- **Data structure** = the way geometric and non-geometric data of a model are organised in computer memory so they can be stored, searched and modified efficiently.
- Information stored for a model:
  - **Geometric data** – coordinates of points, equations of lines / curves / surfaces.
  - **Topological data** – how entities connect (vertex – edge – face relations).
  - **Non-graphic data** – material, colour, layer, dimensions, tolerances, weight, cost.

Common structures

- **Array / table** – fixed list of coordinates; simple but not flexible.
- **Linked list** – each record holds data + a pointer to the next; easy insertion / deletion.
- **Tree (hierarchical)** – parent–child structure, e.g. assembly → sub-assembly → part → feature (CSG tree).
- **Network / graph** – nodes joined in many-to-many manner (B-rep: V, E, F).
- **Relational tables** – data in rows and columns joined by keys.
- A good structure should give fast access, easy modification, small memory and data integrity.

9. Database Management System (DBMS)

- **Database** = organised collection of related data stored once and shared by many users / programs (CAD, CAM, CAPP, MRP). **DBMS** = software that creates, stores, retrieves, updates and protects it.
- Functions: data definition, storage, retrieval, security, integrity, backup & recovery, multi-user concurrency control.
- Advantages: reduced redundancy, data consistency, sharing, standards, security, data independence.

Database models

Model	Structure	Remark
Hierarchical	tree: one parent, many children	fast, rigid; assemblies / BOM
Network	many-to-many links (pointers)	complex but flexible
Relational	tables of rows & columns joined by keys	most popular; SQL queries
Object-oriented	data + methods in objects	suits CAD features, complex geometry

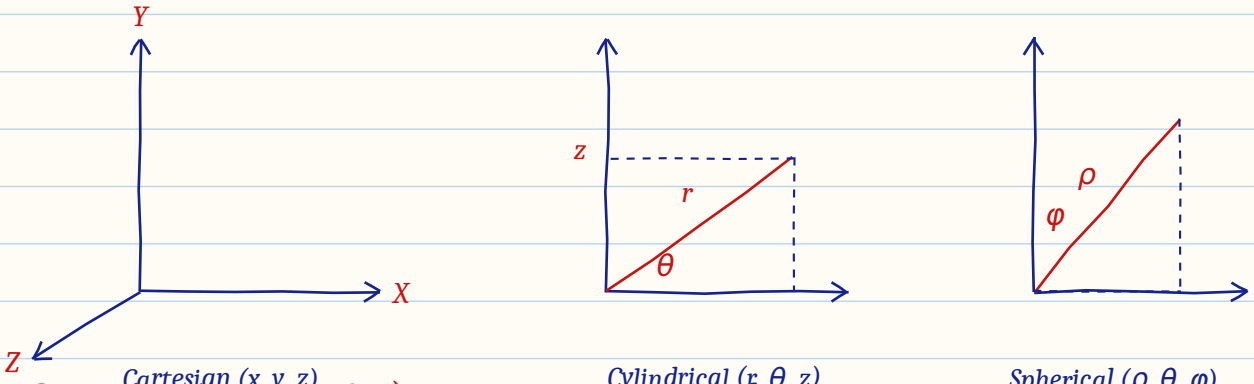
- **Schema** = overall design of the database; **sub-schema** = a user's view; **DDL** defines data, **DML** manipulates data.
- Four requirements for CAD databases: handle complex geometry, version control, long transactions, and links to graphic & non-graphic data.

Computer Graphics

10. Coordinate Systems

- A coordinate system gives numbers (coordinates) to the position of a point in space.
- Three systems are used in a graphics package:
  - **Model (world / master) coordinate system (MCS / WCS)** – the part is defined in real-world units (mm); origin chosen by designer.
  - **Working coordinate system (WCS)** – a temporary user-defined system, placed at convenient position / orientation (e.g. on a face) to draw 2D features on it.
  - **Screen (device) coordinate system (SCS)** – display pixels, origin at screen corner; units are pixels.
- Mapping: world → (viewing transformation) → normalised device coordinates (0 to 1) → (workstation transform) → screen pixels.

Types of coordinate systems in space



Cartesian (x, y, z) Cylindrical (r, θ, z) Spherical (ρ, θ, φ)

- **Cartesian (rectangular)**: x, y, z mutually perpendicular. Right-handed system is standard.
- **Cylindrical**: $x = r\cos\theta$, $y = r\sin\theta$, $z = z$.
- **Spherical**: $x = \rho\sin\varphi\cos\theta$, $y = \rho\sin\varphi\sin\theta$, $z = \rho\cos\varphi$.
- **Polar** (2D): (r, θ) – used in NC programming for circular patterns.

11. Graphics Package Functions

- Graphics package = software library to create, display and manipulate pictures (e.g. GKS, PHIGS, OpenGL, DirectX).
- Standard graphics (GKS – Graphical Kernel System; PHIGS) give device independence and portability.
- Main functions:
  - **Output primitives**: point, line, polyline, circle, arc, text, polygon fill – to generate the picture.
  - **Attributes**: colour, line type & width, text size & font, fill style.
  - **Geometric transformations**: translation, scaling, rotation, reflection.
  - **Viewing functions**: window, viewport, zoom, pan, projections.
  - **Segmenting / structuring**: grouping primitives into named pictures that can be edited.
  - **Input functions**: locator, pick, choice, valuator, string, stroke (interactive devices).
  - **Control & clipping**: initialise workstation, clear screen, clipping against window.
  - **Hidden line / surface removal, shading, colouring** for 3D realism.

Window and viewport

- **Window** = region of the world (model) space to be displayed. **Viewport** = region of the screen where it is shown.

$$xv = xvmin + (xw - xwmin)\cdot sx\,,\quad yv = yvmin + (yw - ywmin)\cdot sy$$

- Scale factors $sx = (xvmax - xvmin)/(xwmax - xwmin)$, $sy = (yvmax - yvmin)/(ywmax - ywmin)$.
- Zooming = shrinking the window; panning = moving the window.

12. Transformations – why needed

- Transformations change the position, size or orientation of an object **without redrawing** it.
- Basic: **translation, scaling, rotation** (+ reflection, shear). Used in modelling, assembly, animation and viewing.
- All are done by multiplying the point coordinates by a matrix. To treat translation also as multiplication we use **homogeneous coordinates**: a point (x, y) is written as $(x, y, 1)$ and a 3D point as $(x, y, z, 1)$.

13. 2D Transformations (homogeneous 3 × 3 matrices)

(a) Translation – move by (tx, ty)

$$x' = x + tx\,,\quad y' = y + ty$$

$$[x'\ y'\ 1] = [x\ y\ 1]\cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ tx & ty & 1 \end{bmatrix}$$

(b) Scaling – by sx, sy about the origin

$$x' = sx\cdot x\,,\quad y' = sy\cdot y$$

$$[S] = \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$sx = sy \rightarrow$ uniform scaling; $sx \neq sy \rightarrow$ differential scaling.

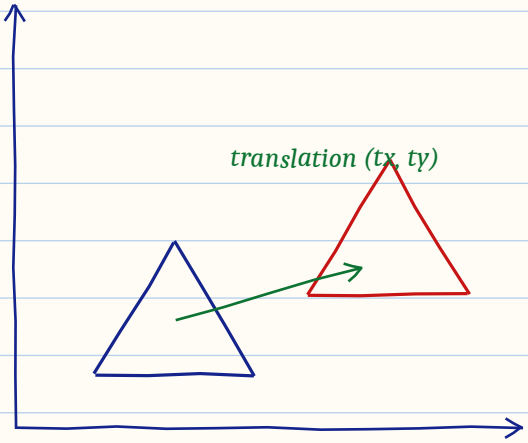
- Scaling about a fixed point (xf, yf): translate (–xf, –yf) → scale → translate (xf, yf).

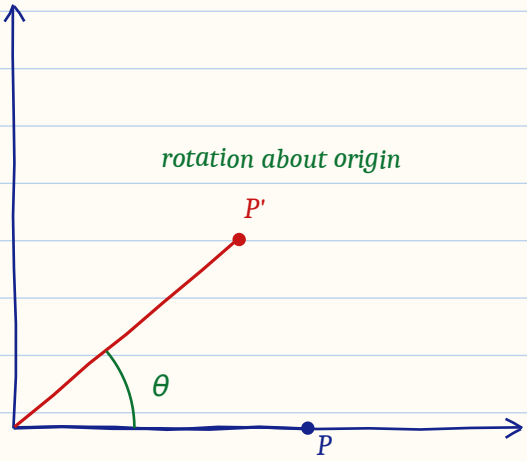
(c) Rotation – by angle θ (anticlockwise +ve) about the origin

$$x' = x\cos\theta - y\sin\theta \;,\; y' = x\sin\theta + y\cos\theta$$

$$[R] = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Rotation about any point (h, k): $T(-h,-k)\cdot R(\theta)\cdot T(h,k)$ (three steps – concatenation).





(d) Reflection (mirror)

- About x-axis: x' = x, y' = –y. About y-axis: x' = –x, y' = y. About origin: both signs change.
- About line y = x: x' = y, y' = x. (the matrix has 1's on the anti-diagonal.)

(e) Shear

- x-shear: x' = x + shxy, y' = y. y-shear: y' = y + shyx, x' = x.

Concatenation of transformations

- Several transformations can be combined into one matrix: M = M1·M2·M3 … (**order matters**, matrix multiplication is not commutative: TR ≠ RT).

Solved example idea: rotate a triangle A(1,1) B(4,1) C(2,3) by 90° about origin → A'(–1,1) B'(–1,4) C'(–3,2).

14. 3D Transformations (4 × 4 homogeneous matrices)

- Point (x, y, z) → (x, y, z, 1); transformation: [x' y' z' 1] = [x y z 1] ·[T].

Translation

$$[T] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ tx & ty & tz & 1 \end{bmatrix}$$

Scaling

$$[S] = \begin{bmatrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & sz & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation about the axes (right-hand rule)

- **About z-axis** (θ): $x' = x\cos\theta - y\sin\theta$, $y' = x\sin\theta + y\cos\theta$, $z' = z$.
- **About x-axis** (θ): $y' = y\cos\theta - z\sin\theta$, $z' = y\sin\theta + z\cos\theta$, $x' = x$.
- **About y-axis** (θ): $z' = z\cos\theta - x\sin\theta$, $x' = z\sin\theta + x\cos\theta$, $y' = y$.
- Rotation about an arbitrary axis: translate the axis to the origin → rotate it to coincide with a coordinate axis (two rotations) → rotate by θ → reverse the two rotations → translate back.

Reflection in 3D

- About xy-plane: z → –z; about yz-plane: x → –x; about zx-plane: y → –y.

Projections (3D → 2D screen)

- **Parallel**: projectors parallel – orthographic (front, top, side views) and axonometric (isometric etc.). Keeps true size & parallelism.
- **Perspective**: projectors meet at the centre of projection (eye) – gives realistic view; far objects look smaller; one-, two-, three-point perspective.

15. Clipping

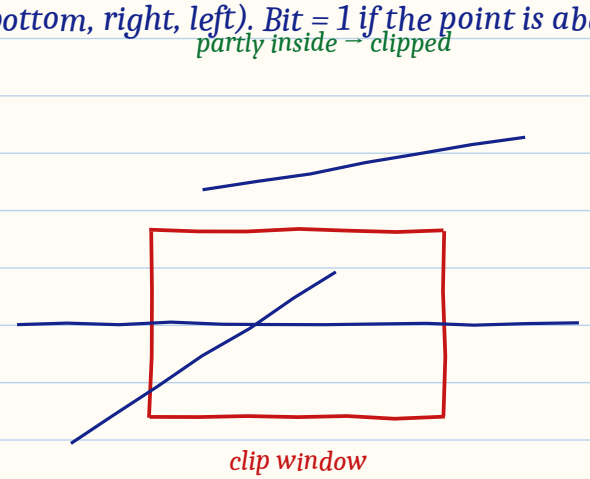
- **Clipping** = removing the parts of the picture lying outside the window (or view volume) so only the visible part is displayed / processed.
- Needed to save computing time and avoid drawing outside the screen area.
- Types: point clipping, line clipping, polygon clipping, text clipping, 3D clipping.

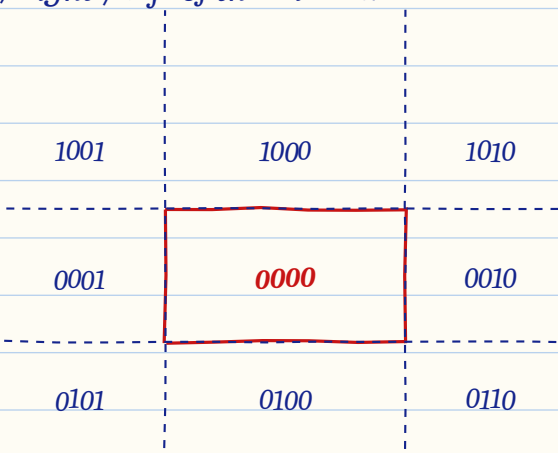
Point clipping

- A point (x, y) is visible if xwmin ≤ x ≤ xwmax and ywmin ≤ y ≤ ywmax.

Cohen–Sutherland line clipping algorithm

- Divide the plane into 9 regions; give each end-point a **4-bit outcode** (bit order: top, bottom, right, left). Bit = 1 if the point is above / below / right / left of the window.





- Steps: (1) find outcodes of both ends P1, P2.
  - If both codes are 0000 → line fully **inside** → **accept**.
  - If (code1 AND code2) ≠ 0000 → fully **outside** on same side → **reject**.
  - Otherwise line is partly inside: pick an outside end, find its intersection with a window edge, replace the end with it, recompute outcode and repeat.
- Intersection formulas (slope m = (y2 − y1)/(x2 − x1)): with left/right edge x = xw → y = y1 + m(xw − x1); with top/bottom edge y = yw → x = x1 + (yw − y1)/m.

Other methods

- **Midpoint subdivision** – repeatedly divide the line at its midpoint and test each half (binary search for intersection).
- **Liang–Barsky** – parametric line P = P1 + u(P2 − P1), 0 ≤ u ≤ 1; clip with inequalities pu ≤ q for each edge – faster, fewer intersections.
- **Sutherland–Hodgman polygon clipping** – clip the polygon against one window edge at a time (left, right, bottom, top); four cases for each edge of the polygon: in→in, in→out, out→out, out→in.

16. Hidden Line / Hidden Surface Removal

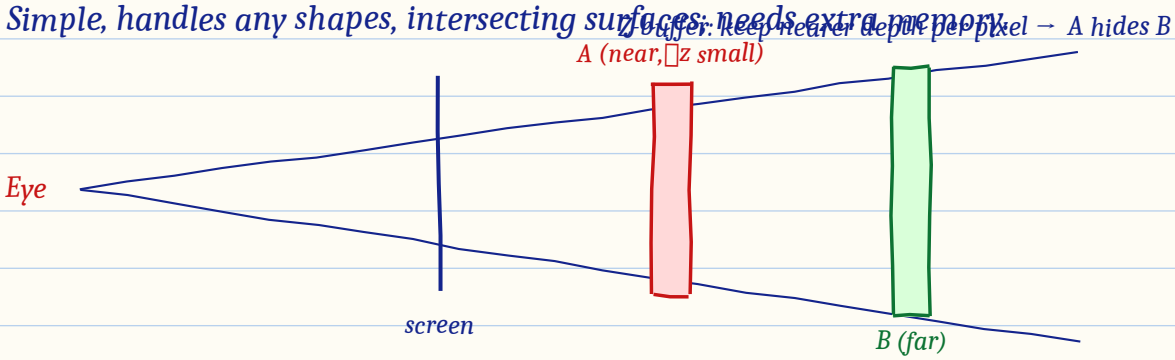
- When a 3D object is shown on a 2D screen, edges / faces that are behind other faces must not be shown, otherwise the picture is ambiguous. The process is **hidden line (HLR) / hidden surface removal (HSR)**.
- Two classes of algorithms:
  - **Object-space methods** – compare objects / faces with each other (accurate, device independent) e.g. back-face removal, depth sorting, Roberts algorithm.
  - **Image-space methods** – decide visibility pixel by pixel on the screen, e.g. Z-buffer, scan-line, Warnock's area subdivision.

Back-face removal (culling)

- A polygon is **back-facing** if its outward normal N makes an angle > 90° with the viewing direction V, i.e. **$N \cdot V > 0$** (when V points from eye to object). Such faces are discarded (removes about half the faces of a closed solid).
- Works only for convex solids; for concave objects other tests are also needed.

Z-buffer (depth-buffer) algorithm

- Keep two arrays: **frame buffer** (colour) and **Z-buffer** (depth of each pixel). Initialise depth = far, colour = background.
- For each polygon and each pixel it covers, compute depth z; if z is nearer than the stored value → update depth and colour; else ignore.
- Simple, handles any shapes, intersecting surfaces; needs extra memory.



Painter's (depth-sort) algorithm

- Sort polygons by depth from farthest to nearest and paint them in that order – nearer faces overwrite farther ones.

Scan-line and Warnock algorithms

- **Scan-line**: process one horizontal line at a time; for each span find the nearest polygon.
- **Warnock**: recursive subdivision of the window until in each area the visible face is

obvious (one polygon, none, or the nearest).

Hidden line methods

- Wire-frame display: edges tested against faces; **Roberts algorithm** – for convex polyhedra; removes back faces, then edges hidden by other faces.

17. Colour and Shading

Colour models

- **RGB** (additive – used on screens: red, green, blue; each 0 to 255).
- **CMY / CMYK** (subtractive – used in printing: cyan, magenta, yellow, black).
- **HSV / HLS** – hue, saturation, value – intuitive for user selection.
- Colour depth: 8-bit = 256 colours, 24-bit "true colour" = 16.7 million; colour look-up table (LUT) reduces memory.

Shading / illumination models

- **Shading** = calculation of light intensity on each point of a surface to give a 3D realistic look.
- Light components: **ambient** (uniform background light), **diffuse** (Lambert's cosine law, $I = Id kd \cos\theta$, rough surfaces), **specular** (shiny highlight, Phong term $ks \cos^n \alpha$).

$$I = Ia \cdot ka + Id \cdot kd \cdot (N \cdot L) + Is \cdot ks \cdot (R \cdot V)^n$$

Shading methods for polygon meshes

Method	Idea	Remark
Flat (constant)	one intensity for the whole polygon	fast, faceted look
Gouraud	intensity at vertices, linear interpolation across polygon	smooth; may miss highlights
Phong	interpolate normals, compute intensity at each pixel	best quality, slow

- **Ray tracing** – follows rays from eye through each pixel to the scene; gives reflection, refraction, shadows (photo-realistic but slow). **Rendering** = shading + shadows + texture + transparency.

18. Quick Revision – Important Points

CAD → design side | CAM → manufacturing side | CIM → whole enterprise on one database.

- *Product cycle: design cycle + manufacturing cycle; CAD/CAM links them.*
- *CAD functions: geometric modelling, engineering analysis, design review, drafting.*
- *CAM areas: computer monitoring & control and manufacturing support.*
- *Data structures: array, linked list, tree, network, relational; DBMS models: hierarchical, network, relational, object-oriented.*
- *Coordinate systems: MCS, WCS, SCS; Cartesian, cylindrical, spherical.*
- *Transformations use homogeneous coordinates; concatenation order matters (TR ≠ RT).*
- *Clipping: Cohen–Sutherland (outcodes), Liang–Barsky, Sutherland–Hodgman.*
- *HSR: back-face, Z-buffer, painter's, scan-line, Warnock. Shading: flat, Gouraud, Phong.*

Likely exam questions

- *Explain the product cycle and the role of CAD / CAM / CIM in it.*
- *Differentiate between CAD, CAM and CIM; list benefits of CIM.*
- *Discuss the criteria for evaluating a CAD/CAM system.*
- *What is a DBMS? Explain the different database models used in CAD.*
- *Derive the 2D rotation matrix; explain rotation about an arbitrary point.*
- *Explain the Cohen–Sutherland line clipping algorithm with a neat sketch.*
- *Explain any two hidden surface removal algorithms. Compare Gouraud and Phong shading.*